Documentação do Projeto: FastPilot

1. Visão Geral do Projeto

O **FastPilot** é um aplicativo multiplataforma de telemetria e acompanhamento de dados da Fórmula 1 em tempo real e de forma histórica. Desenvolvido para Android, iOS e Desktop (JVM) utilizando as tecnologias mais modernas do ecossistema Kotlin.

Seu objetivo é extrair dados profundos e fornecer uma visualização rica sobre o desempenho dos pilotos, clima das sessões, transmissões de rádio e comparações diretas de performance (Duels), apoiando-se em arquitetura moderna e reativa.

---

2. Tecnologias e Bibliotecas (Stack)

A aplicação faz uso do **Kotlin Multiplatform (KMP)** acoplado ao **Compose Multiplatform** para garantir 100% de compartilhamento de regras de negócios e Interface de Usuário.

Core & UI

- **Kotlin**: `2.1.0`
- **Compose Multiplatform**: `1.7.3` para renderização gráfica de UI nativa multi-plataforma.
- **Voyager**: `1.1.0-beta03` - Gerenciamento de navegação fluida, transições visuais e gerenciamento de Tabs (TabNavigator).
- **Vico**: `2.0.1` - Biblioteca para criação de gráficos (Charts) de telemetria e comparação.

Arquitetura & Injeção de Dependência

- **Koin**: `4.0.1` - Injeção de dependência leve e multiplataforma. Conecta as bases do aplicativo desde o Ktor, SQLDelight até a injeção nativa de ViewModels no Compose (`koin-compose-viewmodel`).

Networking & Serialização

- **Ktor Client**: `3.0.3` - Para consumo de requisições HTTP em todas as plataformas (utilizando os engines genéricos `OkHttp` no Android, `Darwin` no iOS e `CIO` no Desktop). O HTTP Client possui Content Negotiation, Timeout Configs e Logging.
- **Kotlinx Serialization**: `1.7.3` - Para transformar e mapear as respostas cruas de JSON para data classes estruturadas, incluindo parse tolerante a chaves desconhecidas.

Persistência de Dados (Offline-First)

- **SQLDelight**: `2.0.2` - Banco de dados embutido via SQLite que gera código Kotlin tipo-seguro a partir de queries nativas em SQL. É utilizado para estratégia de Cache e modo off-line da aplicação.

Assincronismo e Estado Reativo

- **Kotlinx Coroutines**: `1.9.0` - O núcleo da reatividade. ViewModels assíncronos e StateFlow garantem estado de tela consistente (UDF - Unidirectional Data Flow).

Autenticação & Infraestrutura

- **Firebase Auth**: Através do wrapper não-oficial e robusto `dev.gitlive:firebase-auth` multiplataforma, acompanhado de `Play Services Auth` no Android para o botão nativo do "Google Sign-in" e autenticação via Splash + Login Screen.
- **Napier**: Gerenciamento de logs consolidados de debug, warning e error.

---

3. APIs Externas Consumidas

A base de dados se apropria de dois enormes agregadores gratuitos para o ecossistema de F1:

A. OpenF1 API (Real-Time & Telemetria)

- **Endpoint Base:** `https://api.openf1.org/v1`
- **Utilidade:** Serve como o motor de tempo real do aplicativo.
- **Dados mapeados (Endpoints Integrados):**
  - **Sessions & Meetings**: Calendário detalhado.
  - **Drivers**: Informações de pilotos participantes de cada evento.
  - **Car Data (Telemetria)**: O "núcleo duro". Retorna rotações, velocidade, câmbio do carro em microssegundos de intervalo.
  - **Laps & Stints**: Duração das voltas até quebra por milissegundos e ciclo de vida do set de pneus.
  - **Position & Intervals**: Acompanhamento dinâmico quadro-a-quadro e distanciamento temporal do grid.
  - **Race Control**: Avisos diretos da FIA como bandeiras amarelas, safety cards, etc.
  - **Weather**: Temperatura ambiente, asfalto, vento e umidade medidos a cada minuto.
  - **Team Radio**: Links com as faixas de áudio cruas que saem dos rádios das equipes ("Bono, my tyres are dead").
  - **Pit Stops & Location**: Tempos de parada e tracking geo-espacial dos carros.

B. Jolpica API / Ergast API (Agregador Histórico)

- **Endpoint Base:** `https://api.jolpi.ca/ergast/f1/`
- **Utilidade:** Como o serviço clássico do Ergast F1 foi depreciado, o Jolpica funciona como retrocompatibilidade para alimentar contexto macro-histórico do aplicativo.
- **Dados mapeados:**
  - Resumos de Temporada, Tabelas de Corridas históricas.
  - Campeonatos anuais e Standings (Piloto e Construtores/Equipes).
  - Pontuações totais das temporadas.

---

4. Arquitetura da Aplicação (MVI / MVVM + Clean Architecture simplificada)

O sistema foi montado utilizando módulos segregados no gradle e arquitetura estratificada em camadas limpas lógicas na parte do `shared`:

Camada 1: Data e Networking

- Controlada pelo `HttpClientFactory`, onde o Ktor processa interceptadores de cache e timeouts (`HttpTimeout` blindado contra `SocketTimeoutExceptions` oriundos da OpenF1 API).
- Classes como `OpenF1Api.kt` encapsulam as funções de suspending calls cruas de rede.
- O `SQLDelight` atua gerando o `FastPilotDB`, armazenando informações para acesso subjacente sem conexão com a VPN e viabilizando a estratégia híbrida de Load.

Camada 2: Repositories

- Cada bloco de funcionalidade lógica assina um Repository (e.g. `SessionRepository`, `TelemetryRepository`).

- O fluxo comum consiste no fallback dinâmico: ele sempre confere um cache (`FastPilotDB`), e se esgotado o TTL (Time-To-Live), requisita do `OpenF1Api` , injeta no banco assincronamente e cospe os dados atualizados formatados.

Camada 3: ViewModels / Estado de Apresentação (MVI/MVVM)

- O estado é garantido pela exposição exclusiva e blindada via `StateFlow` e imutabilidade dos objetos `UiState` (e.g. `DashboardUiState`).
- O `viewModelScope.launch` controla toda coleta e emissão paralela (assíncrona). Regras e filtros de exceções de concorrência (`CancellationException`) são tratadas minuciosamente para evitar vazamentos de crash invisíveis em rápidas trocas de abas.
- Injeção feita via Módulo do Koin (`koin-compose-viewmodel`).

Camada 4: UI (Telas e Componentes - Voyager)

As abas interativas implementadas no aplicativo pelo `composeApp` :

1. **Splach / Auth Login**: Bloqueio de sessão inicial gerenciados sob Firebase GitLive com token de sessão ativo.
2. **Dashboard**: Central com Live Polling atualizando ativamente o estado global. Transita automaticamente para polling contínuo (modo AO VIVO) em fins de semana de corrida.
3. **Telemetry & Duel**: Permite plotagem densa. Seleciona o piloto 1 e piloto 2 lado-a-lado usando *GetDriverDuelUseCase*. Visualiza delta-time, velocidade por metro.
4. **Team Radio**: Leitor customizável assíncrono para os audios disparados pelos chefes de equipe.
5. **Weather & Strategy**: Relatório dos compostos e mapa meteorológico interativo extraídos a longo prazo das sessões do ano de 2024 a 2025 (e adaptado as limitações do servidor real-time do OpenF1).

Gerado por Antigravity AI Assistant.